MINI PROJECT REPORT

Virtual Teach Pendant Trainer for Industrial Robot Programming in Virtual Reality

MTE 4442 Augmented Reality and Virtual Reality

Submitted by

Sriprada Bhat 230929312

Giridhar Pai 230929002

Nakhuda Mohammed Yusuf 230929320

Mohammed Zayn 230929186

Under the guidance of **Dr Asha C S**

Department of Mechatronics Engineering

Manipal Institute of Technology

Manipal Academy of Higher Education, Manipal

November 2026

Acknowledgement

We thank our course instructor for the guidance given through the project and for the feedback at the fifty percent review, which led directly to the rebuild of the work cell geometry described in Chapter 4. We also acknowledge Future Robotics and Automation Pvt. Ltd., Bengaluru, where one of us worked with a FANUC robot and its teach pendant during industrial training, an experience that shaped the design of this trainer. The project uses Unity 6 and its Universal Render Pipeline, the XR Interaction Toolkit and OpenXR, and the Python libraries pandas, matplotlib and scipy for analysis. All of these are acknowledged as third party tools.

Abstract

Industrial robots are programmed on the shop floor with a handheld teach pendant, but a cell costs several lakhs of rupees and a beginner who jogs the arm the wrong way can damage the arm or the fixture, so engineering students rarely get to practise. This project builds a virtual reality trainer in Unity 6 that reproduces the teach pendant workflow and, unlike existing virtual tools, measures what the trainee does. A six axis articulated robot with the joint axes, travel limits and speeds of a FANUC LR Mate 200iD is placed in a fenced work cell containing a machine table with a fixture, a belt conveyor, a control cabinet and a safety beacon, all generated procedurally in code. The trainee selects a joint, jogs it behind a deadman switch, teaches program points with gripper actions and plays the program back with coordinated joint motion to carry a part from the fixture to the conveyor. The software records every collision between a robot link and the cell hardware, every joint limit violation, the program cycle time, the number of points taught, the total joint travel and whether the task completed, and writes them to CSV without an observer present. A review of 20 papers established that no existing low cost immersive trainer combines the pendant workflow with automatic measurement of trainee performance, which is the gap this work addresses. The trainer is complete and operational as a desktop application, ten system test cases were executed with nine passing, and the measurement and analysis chain was verified end to end. The comparative user study the instrument was built for was designed and prepared but not carried out before submission, and no performance claim is made in its absence.

Table of Contents

Abstract

Acknowledgement

Chapter 1 Introduction
1.1 Background and overview · 1.2 Problem domain · 1.3 Problem statement · 1.4 Scope · 1.5 Significance · 1.6 Organisation

Chapter 2 Literature Review
2.1 Scope and method · 2.2 Foundations · 2.3 VR training for industrial robots · 2.4 Game engines and digital twins · 2.5 Assembly and maintenance training · 2.6 Reviews · 2.7 Comparison of methods · 2.8 Limitations in existing work · 2.9 Relation to this project

Chapter 3 Motivation and Objectives
3.1 Motivation · 3.2 Research gap · 3.3 Need · 3.4 Aim · 3.5 Objectives · 3.6 Expected outcomes and beneficiaries

Chapter 4 Methodology
4.1 Workflow · 4.2 System architecture · 4.3 System design · 4.4 Hardware and software · 4.5 Algorithms and techniques · 4.6 Implementation · 4.7 Work cell · 4.8 Operator feedback · 4.9 Measurement layer · 4.10 Testing method · 4.11 Results · 4.12 Status against objectives

Chapter 5 Individual Contribution

Chapter 6 GitHub Repository
6.1 Link · 6.2 Contents · 6.3 Running the project · 6.4 Configuration

Chapter 7 References

List of Figures

Figure 1. System architecture of the VR teach pendant trainer
Figure 2. Use case diagram
Figure 3. Activity diagram for one training session
Figure 4. User interaction flow in VR
Figure 5. Work cell layout in plan, in metres
Figure 6. Robot joint hierarchy and joint parameters
Figure 7. The six axis arm as built
Figure 8. The completed work cell
Figure 9. The fixture with the part located on its nest
Figure 10. Task completed, part placed on the conveyor
Figure 11. Collisions logged with the link and the object struck
Figure 12. Session counters updating live during a run
Figure 13. A session saved to disk
Figure 14. Output of the analysis script, on simulated data

List of Tables

Table 1. Distribution of the reviewed literature
Table 2. Comparison of the 20 reviewed papers
Table 3. Hardware and software requirements with justification
Table 4. Software modules
Table 5. System test cases and observed results
Table 6. Status of each project objective
Table 7. Repository structure

Chapter 1 Introduction

1.1 Background and overview

Industrial robots are programmed using a teach pendant, which is a handheld controller, on the shop floor by teaching the robot one joint at a time and saving the position as a program. Learning this skill costs a lot of money. The cost of a single six axis arm, its fixtures and safety fencing amounts to several lakhs of rupees, and a beginner who jogs the arm the wrong way can break the arm or the fixture. Therefore, many engineering students get the chance to learn the programming of a robot in theory and meet a real pendant for the first time in industry. Virtual reality offers a way around this. During training, a trainee can stand inside a full scale virtual cell, hold a virtual pendant, make errors without any real consequences, and repeat the same exercise as many times as necessary. There is a second question that this project asks: Can such a trainer be constructed that will measure what the trainee is doing, and determine if the training is effective? The literature already indicates that such a trainer can be constructed, but, in this project, it is asked whether it can be constructed to measure what the trainee does, and to determine if the training is effective. There is also first-hand motivation for this project because of experiences. One team member had the opportunity of completing his industrial training at Future Robotics and Automation Pvt. Ltd., Bengaluru, where he had to work with a FANUC robot using the actual teach pendant and simulate a machine tending cell at FANUC RoboGuide. That experience demonstrated the difference between "simulation" (which uses a mouse to control the robot), and "physical" (pendant control, real robots, real space). The proposed system aims to fill this gap.

1.2 Description of the problem domain

The domain is on line teach pendant programming of serial industrial robots in a machine tending cell. The operator stands beside a fenced cell holding a pendant, selects one joint at a time, jogs it while holding an enabling switch, saves the resulting pose as a program point together with a gripper action, and plays the program back. Mistakes in this domain are physical: the arm strikes a fixture, a joint is driven against its travel stop, or the part is dropped. The cost of those mistakes on real hardware is what prevents beginners from practising.

1.3 Problem statement

A teach pendant programming experience with industrial robots is essential for new engineers, but physical cells are expensive, limited and easy to break. The existing virtual tools either lack immersiveness (web and desktop pendants) or only emphasize control and digital twin accuracy, but not training. The number of systems that automatically capture the trainee's own performance is very small, making it difficult to determine whether the trainee becomes more proficient in programming a robot after a brief amount of VR practice.

1.4 Scope of the project

The scope of this project is a single pick and place task in a single work cell, operated on a desktop computer with a keyboard, with the virtual reality layer prepared but validated only through the XR Device Simulator. Inverse kinematics, multiple tasks, multiple cell layouts and connection to a physical robot controller are outside the scope. The project covers the robot model, the pendant logic, the work cell, the measurement layer, the analysis pipeline and the experimental protocol.

1.5 Significance of the proposed solution

The significance of the proposed solution is that it separates two things the literature usually merges. Reproducing a robot cell in virtual reality is well established. Measuring whether a trainee improves is not, and where it is attempted it is normally done by a person with a stopwatch. A trainer that records the mistakes itself turns an expensive, hard to schedule assessment into a file that is written automatically at the end of every run. The intended beneficiaries are engineering students who have no access to a robot

cell, and training departments that need an objective record of operator competence before a trainee is allowed near real hardware.

1.6 Organisation of the report

The report is organised as follows. Chapter 2 reviews 20 papers on virtual reality for industrial training and identifies the limitations in existing work. Chapter 3 states the motivation, the research gap and the six objectives of the project. Chapter 4 describes the methodology: the system architecture, the design, the hardware and software used, the implementation, the testing carried out and the results obtained. Chapter 5 records the contribution of each team member. Chapter 6 gives the repository link and the instructions needed to run the project. Chapter 7 lists the references.

Chapter 2 Literature Review

2.1 Scope and method

The review focused on VR for industrial training, with robot operation and programming as the main application area. Papers were collected from ScienceDirect, IEEE Xplore, SpringerLink, SAGE, Taylor & Francis, MDPI and Google Scholar using search terms such as virtual reality robot programming training, teach pendant simulation, digital twin industrial robot Unity, and VR assembly training evaluation. Papers from 2021 to 2026 were given priority. Older papers were included only where they are foundational or where they report a directly relevant training experiment. Twenty papers were selected, distributed as shown in Table 1.

Table 1. Distribution of the reviewed literature

Category of research paper	Number of papers	Papers (Table 2 numbering)
Foundational or conceptual studies	3	1 to 3
Recent application-oriented studies	8	4 to 11
Technology and methodology studies	4	12 to 15
Application-specific studies	3	16 to 18
Review or survey papers	2	19 and 20
Total	20	

2.2 Foundations: immersion, presence and comfort

Milgram and Kishino (1994) put real, augmented and fully virtual displays on a reality-virtuality continuum. This project is at the end of the fully virtual range, fitting for it: The whole point is to train where there is no real robot. Slater and Wilbur (1997) distinguished between "immersion" (what the hardware can do) and "presence" (whether or not the user feels like they are there). A presence count is important to a robot trainer because the skill being taught is a spatial one: see how far away the tool is from the fixture from the vantage point you are at. Kennedy et al. (1993) made the Simulator Sickness Questionnaire, which remains the most widely used method to verify the comfort of a VR system. These three works influence two design decisions: walk instead of movement by software and comfort will be considered in addition to performance.

2.3 Does VR training for industrial robots work?

There's some good direct evidence, but also some mixed. In the research presented by Monetti et al. (2022) 24 students conducted drawing and pick and place tasks both on a virtual model and a real ABB IRB 120. The VR group did not require as much time and had more passes. When Pratticò and Lamberti (2021) tested a VR training system and KUKA's own in-class and hands-on training for a maintenance task, the results were the same when the trainees were tested on the actual robot. Both studies are good, but both have small sample sizes and neither requires the trainee to produce a program for the robot.

The most significant result obtained from the project is that of Nathanael et al., 2016. They taught three classes of engineering students on-line robot programming: real robot, plain VR, and VR with cognitive/perceptual support. When students then worked on the real robot, there was no significant advantage given by Plain VR. The use of VR with aids significantly outperformed the real robot group. Similar conclusions were drawn by Manou et al. (2019) from a design perspective, when they added collision-aware perceptual aids to a virtual pick and place scenario, and found improvements in trainee understanding of robot kinematics.

Together these results say that simply copying the real cell into VR is not enough. The trainer has to tell the trainee something the real cell does not, such as when and where they collided.

Wolfartsberger et al. (2023) add an important warning from assembly training. Their non-assisted VR training matched training on the job, but a heavily assisted version, which showed the user exactly what to

do next, led to slower work and more errors afterwards. The lesson is that feedback should report consequences rather than hand out answers. The proposed trainer follows this: it tells the trainee they hit the fixture, but it does not teach the points for them.

2.4 Technology: game engines and digital twins

A common platform for robot VR/digital twin work is now unity. Garg et al. (2021) developed a VR controller to robot program conversion digital twin for FANUC M-10iA/12 by solving the inverse kinematics problem. Singh et al. (2024) linked Unity to ROS and tested the ABB IRB 1200 robot with a mean latency of 77.67 ms and a joint accuracy of 99.99%. Chang et al. (2025) developed a two way virtual-real control platform via a TCP/IP socket link in Unity3D and Pérez et al. (2019) implemented a digital twin control platform with a VR interface for both control and operator training. For their part, Erdei et al. (2022) created a digital twin training centre for a KUKA KR5.

These studies indicate that Unity has the potential and that the rendering, physics and XR support are at a level that are suitable for work with industrial robots. They also display a trend: the work is invested in the fidelity (latency and joint error) to a physical robot. This is a good objective for remote control and commissioning, but it is not the same objective as training. A trainer can not require a real time connection to a live controller. Requires correct kinematic model and good feedback. The physical robot is not in the loop, so is most of the cost, the element of training in VR.

2.5 Lessons from assembly and maintenance training

Assembly and maintenance training has a longer record of controlled experiments. Abidi et al. (2019) found that VR-trained participants completed a real assembly with fewer errors and in less time than traditionally trained ones. Obermair et al. (2024) found VR-trained workers about 10% faster at electric motor wiring with no wiring errors. Gavish et al. (2015), by contrast, found no significant difference between VR training and its control group, probably because their participants were already expert technicians and the task was short. This is a useful caution for experiment design: the participants must be genuine novices, and the task must be hard enough that performance can improve. Kováč et al. (2026) built a VR rehearsal model for end-effector exchange on an industrial robot and were explicit about what it lacked: collision detection, automatic error scoring, time logging and a controlled user study. Hsieh (2019) went the other way, building a web-based virtual pendant with teach and playback that students found useful, but without any immersion.

2.6 What the reviews say

Daling and Schlittmeier (2024) reviewed AR, VR and mixed reality training for manual assembly and found that benefits depend strongly on the device and the task, with measures varying so widely between studies that direct comparison is difficult. Di Pasquale et al. (2024) concluded that VR training for assembly is still limited in practice but promising for learning, productivity and safety. Both reviews call for more controlled studies with objective performance measures.

2.7 Comparison of existing methods

Table 2 compares the 20 papers by objective, methodology, technology, findings and limitations. Rows 1 to 3 are foundational, rows 4 to 11 are recent application studies, rows 12 to 15 cover technology and methodology, rows 16 to 18 are application specific and rows 19 to 20 are reviews.

Table 2. Comparison of the 20 reviewed papers

No.	Authors and Year	Research Objective	Methodology	VR Technology	Key Findings	Limitations
1	Milgram & Kishino (1994)	Classify displays that mix real and virtual imagery	Conceptual taxonomy of mixed reality displays	Head mounted, monitor based and see-through displays in general	Introduced the reality-virtuality continuum that places AR and VR on one scale	Conceptual only; predates consumer headsets and gives no guidance on training design

No.	Authors and Year	Research Objective	Methodology	VR Technology	Key Findings	Limitations
2	Slater & Wilbur (1997)	Define immersion and presence in virtual environments	Conceptual framework (FIVE)	Immersive virtual environments in general	Separates immersion, which the technology provides, from presence, the user's sense of being there; argues presence can affect task performance	Theoretical; no validated measure for a specific task
3	Kennedy et al. (1993)	Create a standard measure of simulator sickness	Factor analysis of symptom reports from flight simulator users	Flight simulators	Produced the 16 item Simulator Sickness Questionnaire with nausea, oculomotor and disorientation subscales, still the standard in VR studies	Built from flight simulator data, not head mounted displays; self report only
4	Pratticò & Lamberti (2021)	Test whether immersive VR can replace company training for industrial robot operators	User study at KUKA: light maintenance task trained in VR versus in-class plus hands-on training; trainees assessed by a company instructor on the real robot	Immersive VR training system with head mounted display	VR training was as effective as the traditional training in making trainees able to complete the task on the real robot	Maintenance task, not programming; authors note room to improve training of interaction with physical tools
5	Garg et al. (2021)	Program a FANUC robot remotely through a VR digital twin	Digital twin of a FANUC M-10iA/12 in Unity; controller position solved with inverse kinematics (BioIK) and sent to the physical robot	Unity, OpenVR headset and controllers	Working VR interface that generates robot programs and runs them on the physical FANUC robot	Demonstrates the programming interface; learning outcomes of novices are not the focus
6	Monetti et al. (2022)	Compare VR training with real robot training for novice operators	24 students performed drawing and pick and place tasks of rising difficulty on a VR model and on a real ABB IRB 120; completion time, pass rate, questionnaire	Unity model, HTC Vive head mounted display	VR users needed less time overall and recorded a higher pass rate; 83% found VR helpful for getting familiar with the real robot	Small student sample and short tasks; performance judged by time and pass rate only
7	Erdei et al. (2022)	Design a digital twin training centre for an industrial robot arm	Digital twin of an educational KUKA KR5 lab so students practise before using the physical robot	Digital twin environment of lab and robot	Safe practice environment intended to reduce accident risk and equipment cost	Singh et al. (2024) note that the performance of this twin was not tested
8	Wolfartsberger et al. (2023)	Check whether VR assembly training outperforms training on the job with a tutor	Two VR variants (assisted and non-assisted) against tutor-led training; assembly time, errors and cognitive load measured	VR training application with head mounted display	VR users reported lower cognitive load; non-assisted VR matched tutor training; heavily assisted VR gave slower assembly and more errors	Manual assembly task, not robot operation
9	Obermair et al. (2024)	Compare VR training with paper instructions for electric motor wiring	22 participants in two groups; cycle times and wiring errors analysed with learning curves	VR training environment	VR group was about 10% faster and made no wiring errors, against two wrong connections in the paper group	Small sample; single assembly task
10	Chang et al. (2025)	Reduce the difficulty of industrial robot programming using a digital twin platform	Digital twin control platform with TCP/IP socket link for real-time state reading and command sending	Unity3D digital twin with virtual-real bidirectional mapping	Demonstrated two way mapping between virtual and physical robot for monitoring and control	Centred on control and monitoring rather than on measuring trainee learning
11	Kováč et al. (2026)	Build a VR model for rehearsing end-effector exchange on an industrial robot	CAD in SolidWorks, lab in SketchUp, visuals in Twinmotion, integration in Pixyz Review; laboratory feasibility check	Oculus Rift, Pixyz Review	Workflow shown to be feasible; redesigned flange cut fastening elements by 66.7%; exchange sequence executable in VR	No collision detection, automatic error scoring, time logging, controlled user study or transfer test
12	Nathanael et al. (2016)	Develop and evaluate a VR trainer for on-line robot programming	Three groups of engineering students: real robot only, plain VR, VR with cognitive and perceptual aids; all tested on the real robot	VR training environment with added cognitive aids	Plain VR gave no significant benefit on the real robot; VR with aids clearly outperformed the real robot only group	Entry level task and student sample

No.	Authors and Year	Research Objective	Methodology	VR Technology	Key Findings	Limitations
13	Manou et al. (2019)	Help trainees understand robot programming through a virtual environment	Virtual arm in pick and place scenarios; teach pendant style interface on a PC keyboard; joint and tool frame motion; perceptual aids with collision detection; trainee questionnaire	Commercial VR authoring platform, keyboard input	Trainees reported better understanding of the robot's capabilities, kinematics and task strategy	Evaluated by questionnaire only; keyboard instead of a handheld pendant
14	Pérez et al. (2019)	Use VR interfaces to control industrial robots and train operators	VR interface driven by a digital twin of the robot cell	Immersive VR interface with robot digital twin	VR offers immersive operator training and robot control without physical risk	Tied to a physical robot and controller, which raises setup cost compared with a standalone trainer
15	Singh et al. (2024)	Build and measure a Unity plus ROS digital twin of a robot arm	Unity model, ROS Noetic, ROS-TCP-Connector and MoveIt; latency and accuracy tested over long cycles on an ABB IRB 1200	Unity 2022.3, ROS, MoveIt	Mean latency of 77.67 ms and joint accuracy of 99.99%	One new robot working alone under standard network conditions; not a training system
16	Gavish et al. (2015)	Evaluate VR and AR training for industrial maintenance and assembly	Expert technicians trained on an electronic actuator assembly with VR, AR or a control condition, then tested on the real task	VR and AR platforms from the SKILLS project	VR and AR groups needed longer training; AR group had fewer unsolved errors; no significant difference between VR and its control group	Likely ceiling effect from expert participants and a short task
17	Abidi et al. (2019)	Assess a VR system for manufacturing assembly training	Participants trained in VR or by a traditional method, then performed the real assembly	VR assembly training system	VR-trained participants completed the real assembly with fewer errors and in less time	Assembly task, not robot operation
18	Hsieh (2019)	Give students remote practice in industrial robot programming	Web-based virtual teach pendant: define path points, save, play back at different speeds; used over several semesters with school and university students	Web browser, 2D virtual pendant	Survey results suggest the virtual pendant helps students learn robot programming	Not immersive; no sense of the robot's space or scale; mainly survey based
19	Daling & Schlittmeier (2024)	Map how AR, VR and MR training affect performance and user ratings in manual assembly	Scoping review	AR projectors, AR handhelds, head mounted displays, VR	Benefits depend on device and task; projector-based AR showed clear gains over paper and video; studies use widely differing measures	Assembly only; mixed study designs limit direct comparison
20	Di Pasquale et al. (2024)	Review VR for assembly and disassembly training	Systematic literature review	Range of VR hardware and software	VR use is limited but promising; can improve learning, productivity and safety while reducing cost	Reports gaps in practical application and calls for further research

2.8 Limitations and gaps identified in existing work

Across the 20 papers, four trends stand out. First, Unity with a head mounted display is the dominant platform. Second, most robot-specific VR work either builds a digital twin validated by technical metrics or trains maintenance and assembly rather than programming. Third, where training effect is measured, the measures are usually completion time, pass rate and questionnaires, collected by an observer. Fourth, the evidence that VR helps depends on the design: plain VR can add nothing, targeted feedback helps, and over-assistance can hurt.

- No immersive trainer puts the real pendant workflow in the trainee's hands. Hsieh (2019) and Manou et al. (2019) create pendant functions on a web page or keyboard, without immersion. Demonstrates a tool dragging (controller) with inverse kinematics to avoid the pendant, immersive, but not intuitive. None provides joint selection, deadman jog, point teaching and playback using a handheld virtual pendant in a full scale cell.

- The system itself is hardly ever used to measure performance. The majority of studies use an observer and a stopwatch. Limited abilities to detect collisions and to score and automatically record timing are cited by Kováč et al. (2026). Objective repeatable data from each run would be available due to automatic logging of collisions, joint limit violations and cycle time.
- Training effect for robot programming in particular is under tested. The controlled comparisons that exist are either about maintenance (Pratticò & Lamberti, 2021), about assembly (Wolfartsberger et al., 2023; Obermair et al., 2024), or use short entry level tasks (Nathanael et al., 2016; Monetti et al., 2022).

2.9 How this project relates to previous work

This project differs from previous work in two respects. First, it puts the actual pendant workflow in the trainee's hands rather than a mouse, a web page or a controller driving an inverse kinematics solver. Second, the system measures the trainee instead of an observer measuring the trainee, which makes the data objective and repeatable and makes a controlled comparison cheap to run. It deliberately does not pursue the fidelity targets, such as millisecond latency and joint error against a live controller, that dominate the digital twin literature, because those are control objectives rather than training objectives.

Chapter 3 Motivation and Objectives

3.1 Motivation for selecting the project

The motivation for this project is first hand. One team member completed industrial training at Future Robotics and Automation Pvt. Ltd., Bengaluru, working with a FANUC robot using the real teach pendant and simulating a machine tending cell in FANUC RoboGuide. That experience showed the distance between simulation driven with a mouse and operation driven with a pendant in a real cell, and it is that distance this project tries to close. The practical problem is that the equipment needed to teach the skill is too expensive and too fragile to let beginners practise on it.

3.2 Research gap being addressed

Research gap: No low cost, immersive VR robot training system that replicates teach pendant programming of a 6 axis industrial robot using a virtual pendant, provides feedback on teach pendant task outcomes (collisions, joint limits, etc), requires the trainee to perform task, but log performance data for comparison of VR practice versus written instruction.

3.3 Need for the proposed solution

The need follows from the gap. A trainer that only looks like a robot cell teaches little, because the literature shows that plain virtual reality adds nothing over conventional training unless it gives the trainee information the real cell does not. A trainer that also records what the trainee did is what makes it possible to answer whether the practice worked, and to answer it with numbers rather than opinion.

3.4 Aim

The aim of the project is to build a low cost immersive trainer that reproduces teach pendant programming of a six axis industrial robot and automatically measures trainee performance, and to use it to compare novices who practise in the trainer against novices given written instructions.

3.5 Objectives

- O1. Immerse a VR work cell in Unity 6 including a six axis articulated robot with realistic joint axes, joint limits and joint speeds based on the FANUC LR Mate 200iD.
- O2. Implement a handheld virtual teach pendant with joint selection, jogging, a deadman switch, speed override, point teaching with gripper actions, program editing and playback.
- O3. Use coordinated joint motion for playback, which is like a real controller: all joints begin and end at the same time and the move time is determined by the slowest joint.
- O4. Design a pick and place operation to get a part from one machine fixture to a conveyor using the trainee's robot.
- O5. Detect and record collisions between arm and cell hardware, joint limit violations, program cycle time, number of taught points and task completion and save to CSV files.
- O6. For novices who practise with the trainer, compare them to novices who practise with only written instructions, following the logged data, reporting the results with appropriate statistics.

3.6 Expected outcomes and beneficiaries

The expected outcomes are a working trainer with a desktop control mode, an automatic per session record of collisions, joint limit violations, cycle time, taught points and task completion, an analysis pipeline that turns those records into group comparisons and learning curves, and an honest statement of whether a short period of practice measurably improves first time robot programming. The intended users are engineering students without access to a robot cell and training departments that need an objective record of operator competence.

Chapter 4 Methodology

4.1 Overall methodology and workflow

The project followed an incremental workflow. The topic was selected and a synopsis submitted, then 20 papers were reviewed and the research gap identified. The architecture and the design diagrams were prepared next, followed by a first working prototype containing the robot model, the pendant logic, the work cell and the logging. That prototype was reviewed at the fifty percent stage, where the feedback was that the cell was not legible because the robot, the conveyor and the fixture were all plain cubes. The geometry was then rebuilt procedurally, two kinematic defects were found and corrected, the CSV export was completed and the measurement chain was verified. Testing was carried out at each stage against the test cases in Table 5.

4.2 Proposed system architecture

The system is divided into five layers (see Figure 1). The trainee and the instructor are situated in the user layer. The hardware layer consists of the headset, the controllers and the PC. The application layer is implemented in C# in Unity 6, where the input handling, the teach pendant logic, the robot model and the gripper are included. The physics in Unity is in the simulation layer, which can detect collisions, and so is the work cell. The feedback and data layer displays the results on the pendant screen and stores them in CSV to be analyzed.

One of the important design considerations is the pendant logic never reads hardware directly. All control signals from the VR controllers and from the desktop keyboard are routed through the same interface. This enables the entire trainer to be written and tested on a laptop and VR can be added without having to rewrite the core.

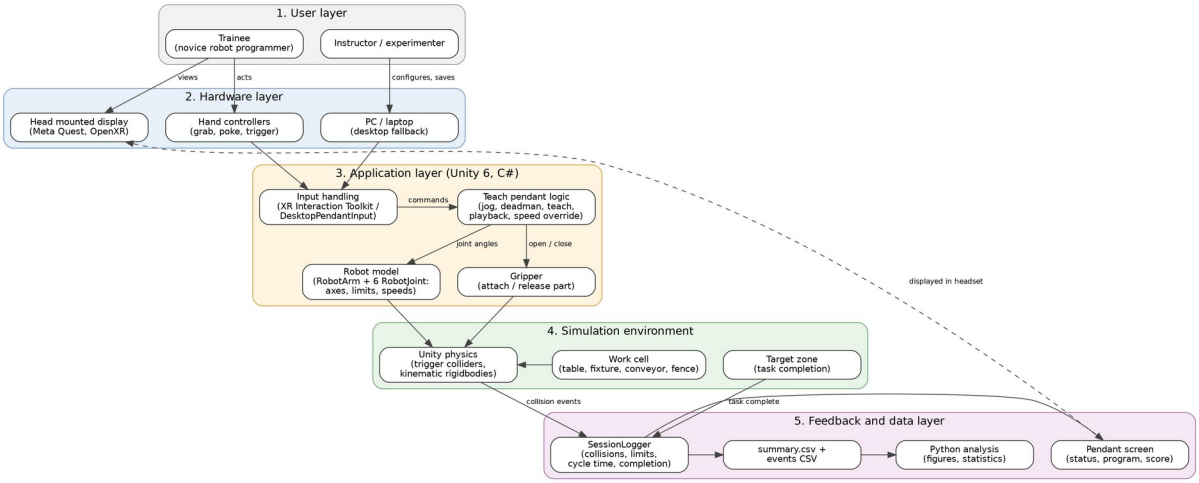


Figure 1. System architecture of the VR teach pendant trainer

4.3 System design and diagrams

The robot is represented as a stack of 6 nested joints J1 to J6, which have only local rotations. Since one joint is a child of the previous joint, rotation of one joint involves all the other joints that follow it. This is the case for a real serial manipulator. J1, J4 and J6 rotate around the vertical axis or the link axis while J2, J3 and J5 rotate around a horizontal axis. The angle limits and maximum speed are stored at each joint. A pose consists of 6 joint angles. In playback the maximum speed for each move is determined by the joint that has the longest path to travel and the time is computed for that joint's maximum speed; all joints are interpolated over this time. This is the same as a J move on a real controller, also it is a joint interpolated move.

In Figure 2, the two actors are depicted. The trainee picks up the pendant, reads the joint selection, jogs the program, teaches points, plays and stops the program, and reads the score. The deadman check and

collision and limit check are always included when jogging. The instructor configures each session, saves the data and analyses the results.

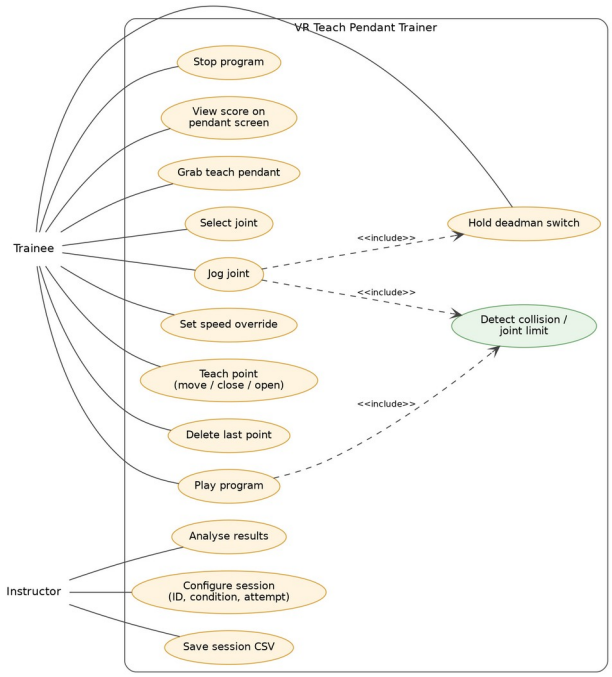


Figure 2. Use case diagram

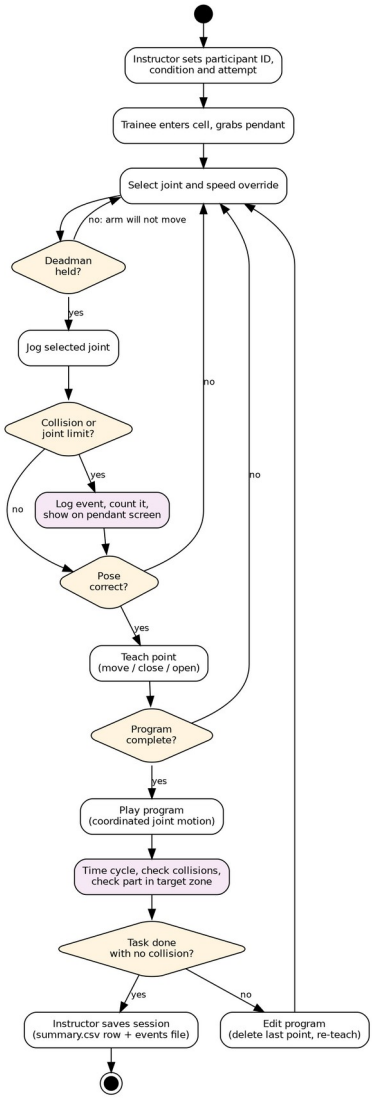


Figure 3. Activity diagram for one training session

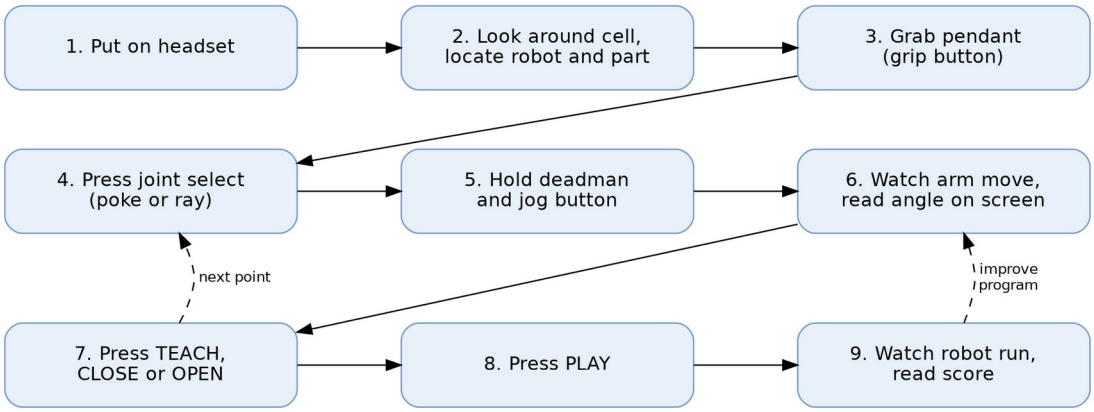


Figure 4. User interaction flow in VR

4.4 Hardware and software requirements, tools and frameworks

Advanced Maths Hardware and Software Requirements are expected to be met.

The requirements are presented in Table 3 along with the rationale for the selection of each option or choice, based on the project goals, available resources, ease of development, compatibility/expected performance.

Table 3. Hardware and software requirements with justification

Requirement type	Proposed requirement	Justification
VR hardware	Meta Quest 2 or 3 with two Touch controllers	Standalone or PC-linked use, inside-out tracking with no base stations, supported by OpenXR; widely available in labs
Development platform	Windows PC or laptop, 16 GB RAM, discrete GPU (GTX 1060 class or better), 30 GB free disk	Enough to run Unity 6 and render stereo VR at a stable frame rate; team laptops meet this
Game engine	Unity 6 (6000.x LTS), Universal Render Pipeline	Specified by the course; URP is designed for good frame rate on headsets; used by Monetti et al. (2022), Garg et al. (2021) and Singh et al. (2024)
Programming	C# (Unity scripting), Python 3 for analysis	C# is Unity's native language; Python with pandas and matplotlib gives quick statistics and figures
3D modelling	Unity primitives and ProBuilder for the prototype; Blender or SolidWorks for refined robot and fixture models	Primitives give a working arm immediately; team has SolidWorks experience for detailed parts later
VR framework	OpenXR with XR Interaction Toolkit 3.x and XR Device Simulator	Vendor neutral; built-in grab, poke and ray interaction; simulator allows testing without a headset
Data logging	CSV files written by the application	Readable in Excel and Python; no database needed
Version control	Git and GitHub with the Unity .gitignore	Required for progress documentation; safe team collaboration
Testing	VR headset, desktop fallback build, 10 to 12 novice participants	Desktop build allows development and demonstration if the headset is unavailable

The software modules written for the project are listed in Table 4. All are C# running under Unity 6. The analysis script is written in Python and uses pandas, matplotlib and scipy. No dataset is used: the project generates its own measurement data at run time.

Table 4. Software modules

Module (C# script)	Responsibility
RobotJoint	One revolute joint: axis, limits, speed, current angle
RobotArm	Ordered joint chain; read and set poses; coordinated move timing
TeachPendant	Jogging, deadman, speed override, teaching, program storage, playback
Gripper	Attach a part to the tool on close, release it to physics on open
LinkCollisionReporter	Reports contact between a robot link and cell hardware
TargetZone	Decides when the part has been placed and settled on the conveyor
SessionLogger	Counts collisions, limits, cycle time; writes CSV files
PendantScreen	Displays status, program list and live score on the pendant
PendantButton	Maps a VR button press on the pendant to a pendant function
DesktopPendantInput	Keyboard control for development and desktop fallback
RobotBuilder (editor tool)	Generates the six joint robot hierarchy with correct axes, limits and colliders

4.5 Algorithms and techniques

The technique underlying playback is joint interpolated motion. For each taught point the controller reads the current pose, asks every joint how long it needs to reach its target at its own maximum speed, takes the longest of those times as the duration of the move, and interpolates all six joints over that duration so that they start and finish together. This is what a real controller does for a J move. The alternative, linear motion, requires an inverse kinematics solver to work backwards from a straight line path to joint angles at every instant, and was deliberately not implemented: joint motion is what a trainee learns first on a real pendant, and an inverse kinematics solver would have consumed the project without contributing to the training question. The consequence, which is also true on a real machine, is that the tool sweeps a curve through space rather than travelling in a straight line.

4.6 Implementation of the robot model

One constraint emerged during implementation that the design had not accounted for. For a straight down pick the wrist centre must sit directly above the part by the whole length of the tool. The wrist centre reaches only 0.590 m from the shoulder, being the sum of the upper arm and forearm lengths. With the robot bolted flat to the floor its shoulder sits at 0.330 m, the part on the machine table sits at 0.760 m, and the wrist would have had to reach 0.680 m, which is 90 mm more than the arm possesses. The part was reachable only with the tool held almost horizontal, which is not how a part is taken out of a fixture. The defect was diagnosed by solving the kinematic chain numerically for each taught position rather than by trial and error. It was corrected by mounting the robot on a 0.400 m fabricated pedestal, which raises the shoulder to 0.730 m, and lowering the machine table top from 0.70 m to 0.60 m. Every taught position now places the wrist between 81 and 87 percent of its reach, which leaves the arm a visible bend rather than working at full stretch. A pedestal of this size is also what a real LR Mate 200iD machine tending cell uses, so the correction improved the realism as well as the kinematics.

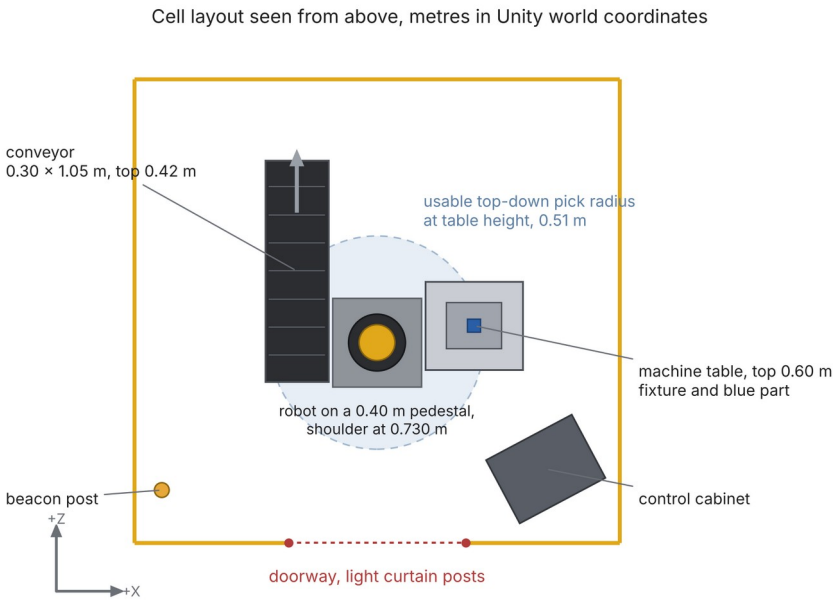


Figure 5. Work cell layout in plan, in metres

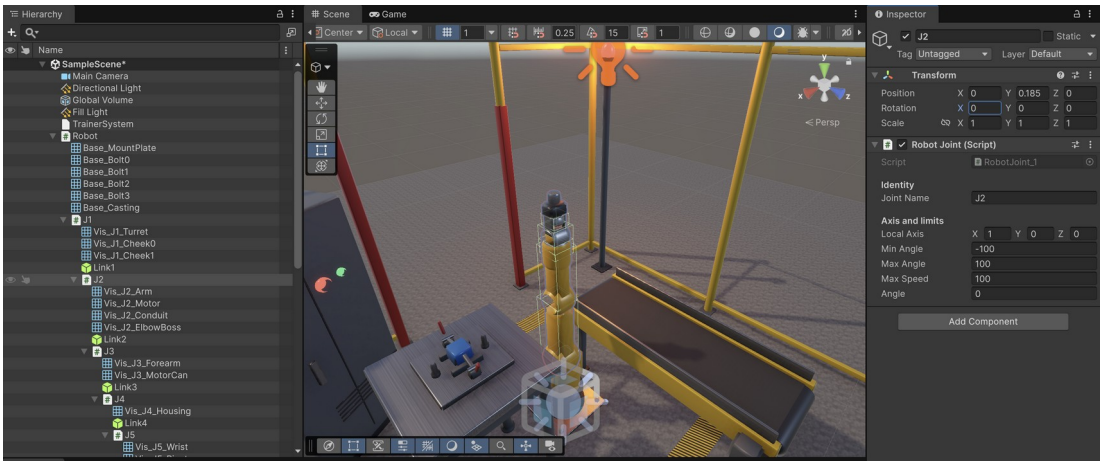


Figure 6. Robot joint hierarchy in the Unity editor, with the joint component for J2 showing its local axis, travel limits and maximum speed

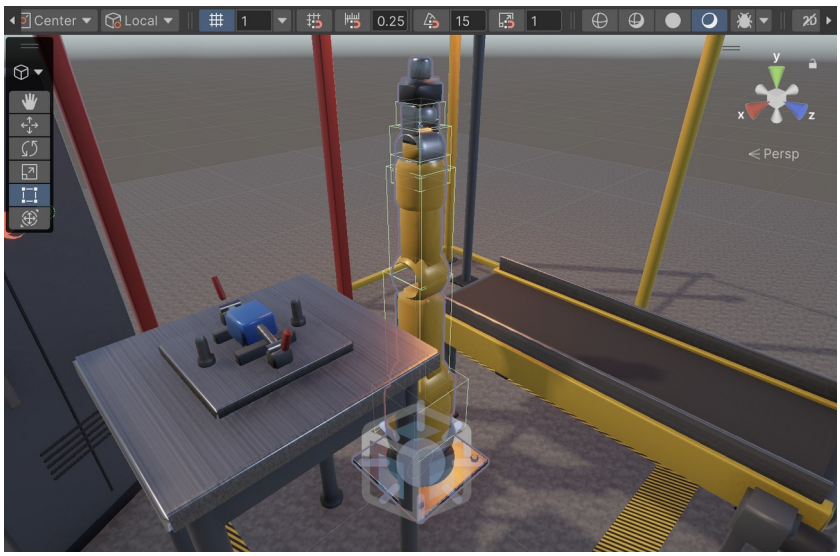


Figure 7. The six axis arm as built, showing the castings, motor barrels, wrist cluster and gripper

4.7 Implementation of the work cell and the gripper

Every shape in the cell is generated by code rather than imported from a model library. Three mesh primitives are used. A lathe revolves a profile of radius and height pairs about a vertical axis, producing the robot castings, the motor barrels, the conveyor rollers, the locating pins, the fence posts and the beacon dome. A bevelled box generates a cube with chamfered edges, which is what stops a machined part reading as placeholder geometry, because a chamfer catches a highlight along every edge where a plain cube does not. A textured quad generates the conveyor belt and the fence mesh panels. The textures are generated the same way from deterministic value noise, so the project renders identically on every machine, and the wire mesh fence uses alpha clipping so that it is genuinely transparent. This approach was adopted after the fifty percent review and has two further advantages for an academic submission: there is no asset licence to account for, and the entire cell regenerates from one editor menu item on any machine.

The gripper does not simulate finger friction, because friction based grasping is unstable in a game physics engine and makes the part jitter. Closing the gripper parents whatever part lies inside its trigger volume to the tool so that the part travels with it, and opening releases the part and returns it to physics. This is how commercial robot simulation packages handle grasping.

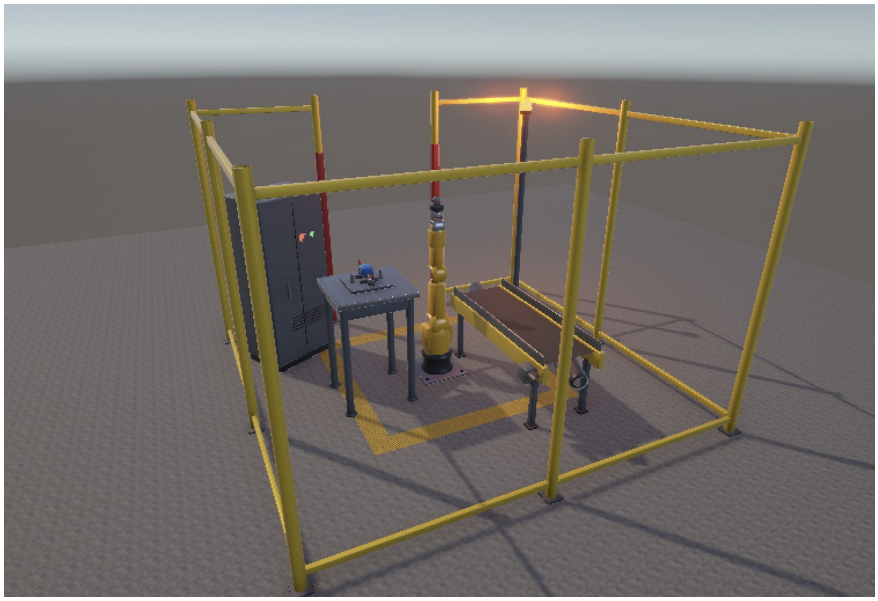


Figure 8. The completed work cell: robot on its pedestal, machine table with fixture, belt conveyor, wire mesh fencing, control cabinet and safety beacon

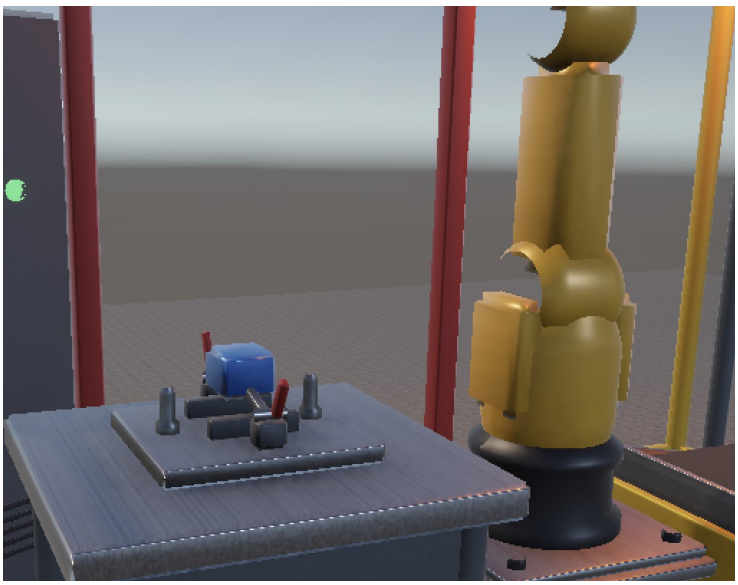


Figure 9. The fixture with the part located on its nest between the locating pins and toggle clamps

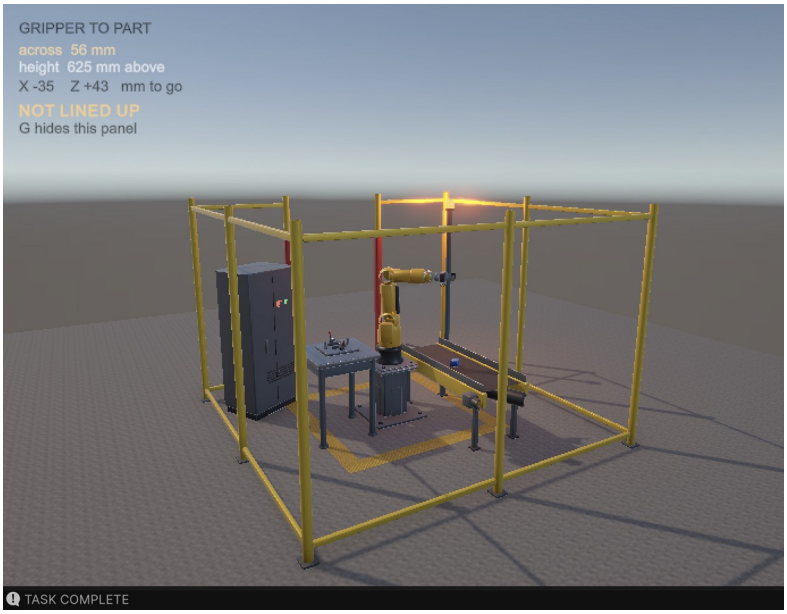


Figure 10. Task completed. The part has been placed on the conveyor and the target zone has reported completion on the status bar

4.8 Operator feedback

During testing it became clear that a trainee cannot judge whether the gripper is above the part from the rendered view, because a perspective camera makes a gripper 80 mm to one side look the same as one that is centred. A real teach pendant displays the tool position numerically for exactly this reason. A readout was therefore added that reports, in millimetres, the horizontal offset and the height of the gripper relative to its target, the signed offset along each axis, and a status line that changes from NOT LINED UP to LINED UP, GO DOWN and finally to IN THE GRIP. Once the part is held, the readout switches to the drop zone above the conveyor. This is feedback on state rather than instruction: it reports where the tool is, as the real machine does, but never indicates which points to teach.

4.9 Measurement layer

Collision detection required a specific arrangement. The arm is moved by setting transform values in code rather than by physics forces, and Unity does not raise ordinary collision events for objects moved that way. Each link therefore carries a trigger collider together with a kinematic rigidbody, because trigger events do fire between a kinematic rigidbody and a static collider. The visible castings carry no colliders at all; collision

uses six simple box volumes named Link1 to Link6, each sized to the space its casting sweeps through. Separating visual geometry from collision geometry is standard practice in commercial robot simulation, it keeps the physics stable, and it meant the collision behaviour was unchanged when the geometry was rebuilt.

Ten quantities are recorded per session and written to summary.csv as one row, together with an event timeline file: the participant code, the condition, the attempt number, the collision count, the joint limit violation count, the program cycle time, the number of points taught, the total joint travel in degrees, whether the task completed, and the elapsed session time. Nothing is recorded by hand, so there is no opportunity for observer bias in the measurement itself.

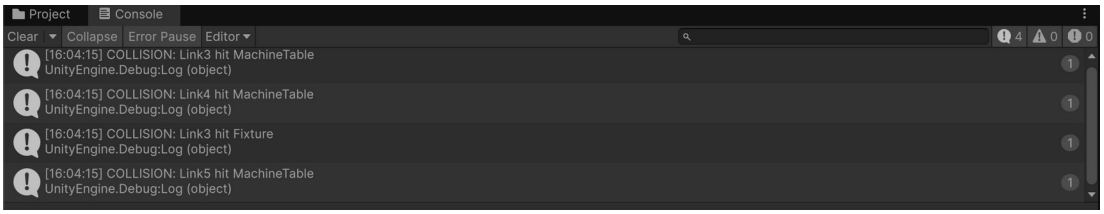


Figure 11. Collisions between individual robot links and the cell hardware logged with a timestamp and the name of the link and the object struck

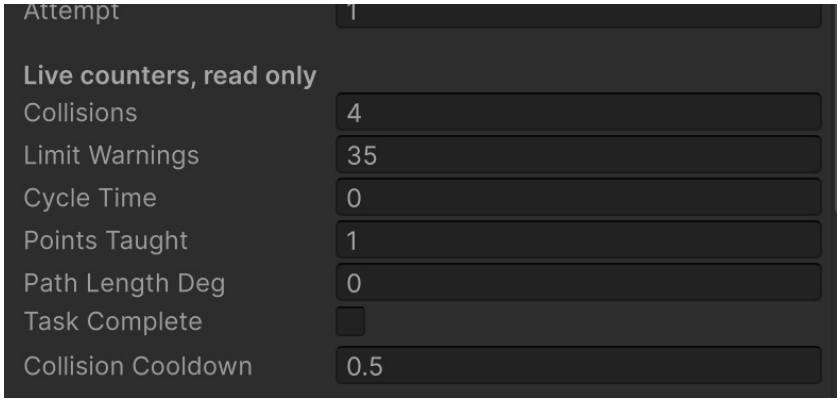


Figure 12. Session counters updating live during a run

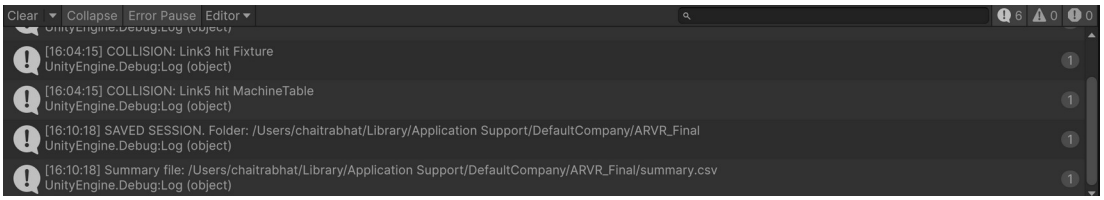


Figure 13. A session saved to disk, with the folder and the full path of summary.csv

4.10 Testing and evaluation method

Testing was carried out in two parts. Functional testing executed the ten cases in Table 5 against the running system, checking each stated expectation against what was observed. Verification of the measurement chain was carried out separately, because a measurement instrument is only useful if the data it writes can be read back and analysed correctly. A dataset of twenty four runs was generated in the exact schema the session logger writes and passed through the analysis script. The values in that dataset are generated, not measured: every row carries a provenance column the logger never writes, and the analysis script detects that column and watermarks every figure it produces. Figure 14 therefore carries that watermark and is included to show the analysis working, not to report an effect.

The comparative user study the instrument was built for was designed in full. Twelve novice participants would be split into two conditions of six: one given five minutes of guided practice in the trainer, the other a single page of written instructions covering the same material. Each participant attempts the same scored task twice, so that improvement across attempts can be measured as well as the difference between conditions. Participants are identified only by sequential codes and no names are recorded. The study was

not carried out before the submission date. Rather than estimate what the result might have been, this report states that it was not conducted and reports only the verification of the instrument. Running it requires one evening and no further development work.

4.11 Results and observations

Nine of the ten test cases passed. The deadman switch stopped the arm immediately on release, joints stopped at their travel limits with the violation counted, collisions between a link and the cell hardware were named and counted, the gripper picked and released the part, the task completion was detected when the part settled on the conveyor, and the session was written to summary.csv and an event file. Test T9, repeatability of the cycle time across repeated playbacks, was not performed before submission and is recorded as not performed rather than estimated.

Table 5. System test cases and observed results

No.	Test	Expected result	Observed result
T1	Hold deadman and jog each joint J1 to J6	Each joint rotates about its own axis and the links beyond it follow	Passed. Each joint rotated about its own axis and the links beyond it followed
T2	Release deadman while jogging	Arm stops immediately	Passed. Arm stopped immediately on release
T3	Jog a joint past its travel limit	Joint stops at the limit and a violation is counted	Passed. 35 limit warnings counted across the pilot session
T4	Drive a robot link into the machine table or the fixture	Collision detected, named in the Console and counted	Passed. Reported per link with the object struck, for example Link3 hit MachineTable
T5	Close the gripper with the part inside the grip volume	Part is picked up and travels with the tool	Passed. Part picked up and carried
T6	Open the gripper above the conveyor	Part is released and falls to the belt	Passed. Part released onto the belt and carried away
T7	Complete the pick and place task	Target zone reports completion once the part settles	Passed. Task completion reported, Figure 11
T8	Save the session	summary.csv and the event file are written to the save folder	Passed. Both files written and the path printed, Figure 13
T9	Play one taught program repeatedly and compare cycle time	Cycle time repeatable within a small tolerance	Not performed before submission
T10	Approach the part using the tool position readout	Readout reports offset and height and indicates when the part is in the grip	Passed. Readout tracked the gripper and changed state on approach

The system was exercised by the team during testing and those sessions were logged. One session recorded 4 collisions and 35 joint limit warnings while the arm was jogged manually towards the fixture, shown on the live counters in Figure 12 and written to disk in Figure 13. The high limit count reflects an operator holding a joint against its travel stop while hunting for a pose, which is precisely the behaviour the trainer is built to surface. In a separate run the task completed with the part placed on the conveyor and the target zone reporting completion, shown in Figure 11. These are developer pilot runs rather than participant data and no performance claim is made from them.

The verification run completed without error on all twenty four generated rows. Group means and standard errors, the learning curve across two attempts, the clean pass rate and Welch two sample t tests for each metric all computed and plotted correctly, and the schema written by the session logger was confirmed field by field against the schema the analysis expects. That last check matters, because a mismatch in a single column name would only have surfaced after participants had already been run.

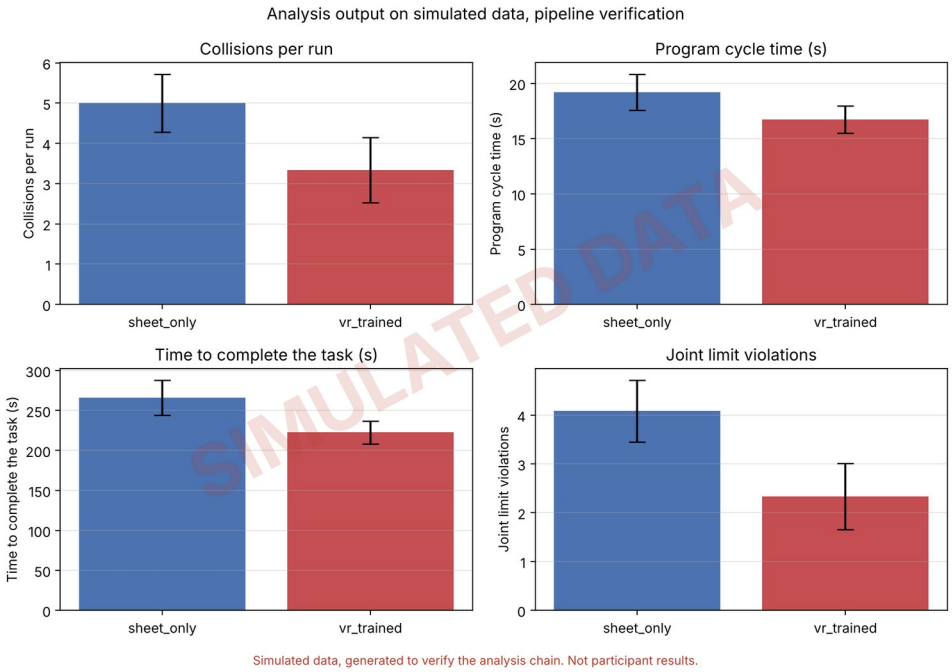


Figure 14. Output of the analysis script. Simulated data, generated to verify the analysis chain. Not participant results

4.12 Status against the objectives

Table 6. Status of each project objective

Objective	Where addressed	Status
O1 Immersive cell with a six axis robot using LR Mate 200iD axes, limits and speeds	Sections 4.4 to 4.6, Figures 5 to 8	Complete as a desktop application
O2 Virtual teach pendant with joint selection, jogging, deadman, speed override, teaching and playback	Sections 4.7 and 4.8, Figure 12	Logic complete and operated from the desktop. The handheld pendant object and headset control were not built
O3 Coordinated joint motion with all joints starting and finishing together	Section 4.5, Table 5	Complete
O4 Pick and place task from a machine fixture to a conveyor	Sections 4.6 and 4.7, Figures 9 to 11	Complete
O5 Record collisions, joint limit violations, cycle time, taught points and task completion and save to CSV	Sections 4.9 and 4.11, Figures 12 and 13	Complete
O6 Compare novices who practise in the trainer against novices given written instructions	Sections 4.10 and 4.12	Not carried out. Protocol, instrument and analysis are complete and verified. No participants were run before submission

Chapter 5 Individual Contribution

The contribution of each team member is recorded below. Every member worked on the code as well as on the documentation, and each is prepared to explain the modules listed against their name.

Sriprada Bhat 230929312

Tasks and responsibilities. Problem definition, objectives and overall project direction; literature review of the robot training and robot programming studies, rows 4 to 11 and 16 to 18 of Table 2; the report.

Modules developed. RobotJoint.cs, RobotArm.cs and TeachPendant.cs: the joint model, the ordered joint chain, pose read and write, the move duration calculation and the playback coroutine.

Deliverables and outcomes. Diagnosed the reach defect described in Section 4.6 by solving the kinematic chain numerically, and specified the pedestal height and table height that corrected it. Wrote Chapters 1, 2, 3 and 7 of this report.

Giridhar Pai 230929002

Tasks and responsibilities. Requirements analysis and technology selection, Table 3; Unity project setup and version control; the GitHub repository.

Modules developed. MeshKit.cs, TextureKit.cs, AssetForge.cs, RobotBuilderPro.cs and CellBuilderPro.cs: the mesh and texture generation and the editor tools that build the robot and the work cell.

Deliverables and outcomes. Rebuilt the cell geometry procedurally after the fifty percent review, set up OpenXR and the XR Interaction Toolkit, and produced the progress screenshots used as Figures 6 to 13.

Nakhuda Mohammed Yusuf 230929320

Tasks and responsibilities. Literature review of the assembly and maintenance training studies and the review papers, rows 1 to 3, 12 to 15 and 19 to 20 of Table 2; system design.

Modules developed. CellSafety.cs, SessionLogger.cs, TargetZone.cs and analyse.py: collision reporting, the session counters, the task completion zone, the CSV export and the analysis script.

Deliverables and outcomes. Prepared the architecture, use case, activity and interaction flow diagrams, Figures 1 to 4. Implemented the CSV export and the provenance watermarking in the analysis script, and wrote Section 4.9 of this report.

Mohammed Zayn 230929186

Tasks and responsibilities. Test planning and execution; experimental protocol and participant materials; verification of the measurement chain.

Modules developed. DesktopPendantInput.cs and PickAssist.cs: the keyboard control mapping and the tool position readout added after testing showed the approach cannot be judged from a perspective view.

Deliverables and outcomes. Executed the ten test cases in Table 5, ran and logged the pilot sessions reported in Section 4.11, and verified the logger schema field by field against the analysis script.

Chapter 6 GitHub Repository

6.1 Repository link

ADD REPOSITORY LINK HERE BEFORE SUBMITTING

6.2 Repository contents

The repository holds the complete Unity project, the C# source, the analysis script and the supporting material for this submission, together with the demonstration video and audio recorded for the project.

Table 7. Repository structure

Path	Contents
Assets/Scripts	The fourteen runtime C# scripts: the joint and arm model, the teach pendant, the gripper, the session logger, the collision reporter, the target zone, the desktop input mapping, the tool position readout, the mesh and texture generation and the cell animation
Assets/Scripts/Editor	The four editor scripts that build the robot, the work cell, the materials and the scene. This folder name is required by Unity and must not be changed
Assets/Scenes	The work cell scene
Packages, ProjectSettings	Unity package manifest and project configuration, tracked so the project opens with the same settings on any machine
analyse.py	The analysis script that reads summary.csv and produces the group comparison, learning curve and clean pass figures
Media	The demonstration video and audio recorded for the project
README.md	Project overview, dependencies and the run instructions reproduced in Section 6.3

The project demonstration video and the accompanying audio are held in the Media folder of the same repository, so the repository link above also gives access to the demonstration.

6.3 Installing dependencies and running the project

1. Install Unity Hub and a Unity 6 editor, version 6000.x LTS, with the Universal 3D template available.
2. Clone the repository and open the project folder from Unity Hub. The first launch takes several minutes while Unity rebuilds its Library folder, which is not tracked in the repository by design.
3. Open Edit, Project Settings, Player, Other Settings and set Active Input Handling to Both. Unity restarts. Without this the keyboard controls do not respond.
4. Open Window, TextMeshPro and choose Import TMP Essential Resources.
5. Open the scene in Assets/Scenes and run Tools, Robot Trainer, 0. Build Everything. This generates the textures and materials, builds the robot and the work cell, sets the lighting, frames the camera and wires the trainer system together.
6. Press Play, click inside the Game view, then hold Space as the deadman and use keys 1 to 6 with the arrow keys to jog each joint. T teaches a point, C and O teach gripper close and open, P plays the program and K saves the session to CSV.
7. For the analysis, copy summary.csv from the save folder next to analyse.py, install pandas, matplotlib and scipy, and run python analyse.py.

6.4 Configuration requirements and access

No API keys, credentials or private data are required or included. The only configuration needed is the Active Input Handling setting in step 3 above. The repository is public. The four editor scripts must remain inside a folder named exactly Editor, which Unity treats specially; moving them elsewhere causes the project to fail to compile.

Chapter 7 References

Abidi, M. H., Al-Ahmari, A., Ahmad, A., Ameen, W., & Alkhalefah, H. (2019). Assessment of virtual reality-based manufacturing assembly training system. The International Journal of Advanced Manufacturing Technology, 105, 3743-3759. https://doi.org/10.1007/s00170-019-03801-3

Chang, W., Sun, W., Chen, P., & Xu, H. (2025). Construction and validation of a digital twin-driven virtual-reality fusion control platform for industrial robots. Sensors, 25(13), 4153. https://doi.org/10.3390/s25134153

Daling, L. M., & Schlittmeier, S. J. (2024). Effects of augmented reality-, virtual reality-, and mixed reality-based training on objective performance measures and subjective evaluations in manual assembly tasks: A scoping review. Human Factors, 66, 589-626. https://doi.org/10.1177/00187208221105135

Di Pasquale, V., Cutolo, P., Esposito, C., Franco, B., Iannone, R., & Miranda, S. (2024). Virtual reality for training in assembly and disassembly tasks: A systematic literature review. Machines, 12(8), 528. https://doi.org/10.3390/machines12080528

Erdei, T. I., Krakó, R., & Husi, G. (2022). Design of a digital twin training centre for an industrial robot arm. Applied Sciences, 12(17), 8862. https://doi.org/10.3390/app12178862

Garg, G., Kuts, V., & Anbarjafari, G. (2021). Digital twin for FANUC robots: Industrial robot programming and simulation using virtual reality. Sustainability, 13(18), 10336. https://doi.org/10.3390/su131810336

Gavish, N., Gutiérrez, T., Webel, S., Rodríguez, J., Peveri, M., Bockholt, U., & Tecchia, F. (2015). Evaluating virtual reality and augmented reality training for industrial maintenance and assembly tasks. Interactive Learning Environments, 23(6), 778-798. https://doi.org/10.1080/10494820.2013.815221

Hsieh, S.-J. (2019). Development and evaluation of remote virtual teach pendant for industrial robotics education. International Journal of Engineering Education, 35(6A).

Kennedy, R. S., Lane, N. E., Berbaum, K. S., & Lilienthal, M. G. (1993). Simulator sickness questionnaire: An enhanced method for quantifying simulator sickness. The International Journal of Aviation Psychology, 3(3), 203-220. https://doi.org/10.1207/s15327108ijap0303_3

Kováč, J., Malega, P., & Vaulin, P. (2026). Development and laboratory feasibility validation of a virtual reality simulation model for robotic end-effector assembly training. Modelling, 7(4), 125. https://doi.org/10.3390/modelling7040125

Manou, E., Vosniakos, G.-C., & Matsas, E. (2019). Understanding industrial robot programming by aid of a virtual reality environment. International Journal of Mechanical Engineering Education. https://doi.org/10.1177/0306419018762704

Milgram, P., & Kishino, F. (1994). A taxonomy of mixed reality visual displays. IEICE Transactions on Information and Systems, E77-D(12), 1321-1329.

Monetti, F. M., de Giorgio, A., Yu, H., Maffei, A., & Romero, M. (2022). An experimental study of the impact of virtual reality training on manufacturing operators on industrial robotic tasks. Procedia CIRP, 106, 33-38. https://doi.org/10.1016/j.procir.2022.02.151

Nathanael, D., Mosialos, S., & Vosniakos, G.-C. (2016). Development and evaluation of a virtual training environment for on-line robot programming. International Journal of Industrial Ergonomics, 53, 274-283. https://doi.org/10.1016/j.ergon.2016.02.004

Obermair, F., Althaler, J., Wolfartsberger, J., Niedermayr, D., & Feichtenschlager, H.-P. (2024). Assembly training for electric motor wiring with virtual reality compared to paper instructions: Experiment and analysis. Procedia CIRP, 130, 398-406. https://doi.org/10.1016/j.procir.2024.10.106

Pérez, L., Diez, E., Usamentiaga, R., & García, D. F. (2019). Industrial robot control and operator training using virtual reality interfaces. Computers in Industry, 109, 114-120. https://doi.org/10.1016/j.compind.2019.05.001

Pratticò, F. G., & Lamberti, F. (2021). Towards the adoption of virtual reality training systems for the self-tuition of industrial robot operators: A case study at KUKA. Computers in Industry, 129, 103446. https://doi.org/10.1016/j.compind.2021.103446

Singh, M., Kapukotuwa, J., Gouveia, E. L. S., Fuenmayor, E., Qiao, Y., Murry, N., & Devine, D. (2024). Unity and ROS as a digital and communication layer for digital twin application: Case study of robotic arm in a smart manufacturing cell. Sensors, 24(17), 5680. https://doi.org/10.3390/s24175680

Slater, M., & Wilbur, S. (1997). A framework for immersive virtual environments (FIVE): Speculations on the role of presence in virtual environments. Presence: Teleoperators and Virtual Environments, 6(6), 603-616. https://doi.org/10.1162/pres.1997.6.6.603

Wolfartsberger, J., Zimmermann, R., Obermeier, G., & Niedermayr, D. (2023). Analyzing the potential of virtual reality-supported training for industrial assembly tasks. Computers in Industry, 147, 103838. https://doi.org/10.1016/j.compind.2022.103838